Online Order Processing Facade

Software Design Patterns Assignment 4

Java 17 implementation | Option B | Individual project report

1 Introduction

This project implements the Facade structural design pattern for a small online shop. A client places an order through one operation, while four services handle inventory, payment, shipping, and confirmation. The central result is a simpler client that does not need to know the checkout sequence or implement its failure recovery.

Without a facade, each client would have to reserve the correct products, obtain the server-calculated total, debit an account, arrange delivery, and send a confirmation. A failure after reservation also requires earlier changes to be reversed. Repeating that coordination in every client would increase coupling and create inconsistent error handling. This is the type of integration problem described in Lecture 6 [2].

2 Components and responsibilities

Component	Responsibility
Main	Application client. Creates the facade and business data; demonstrates checkout.
OrderFacade	Coordinates checkout in order and compensates completed steps after a failure.
InventoryService	Validates the complete cart, calculates its total, and records stock reservations.
PaymentProcessor	Checks account balance, records charges, and refunds a charge at most once.
ShippingService	Validates delivery information, stores shipment records, and cancels shipments.
NotificationService	Checks the email format and stores a composed confirmation in a local outbox.
Customer and OrderReceipt	Java records carrying business input and the successful checkout result.

3 Implementation scope

The sample catalog contains two keyboards at 50.00 USD each and five mice at 25.00 USD each. Alice starts with 100.00 USD and Bob with 10.00 USD. Each facade instance owns a separate shop. All services maintain real in-memory state; they do more than print messages. Money uses BigDecimal. No external libraries or additional design patterns are required.

4 UML class diagram

The diagram shows the Client, the Facade, and its four subsystem classes. The dashed arrow is a usage dependency. Filled diamonds indicate that this implementation creates and owns its service objects. No subsystem has a reference back to the facade.

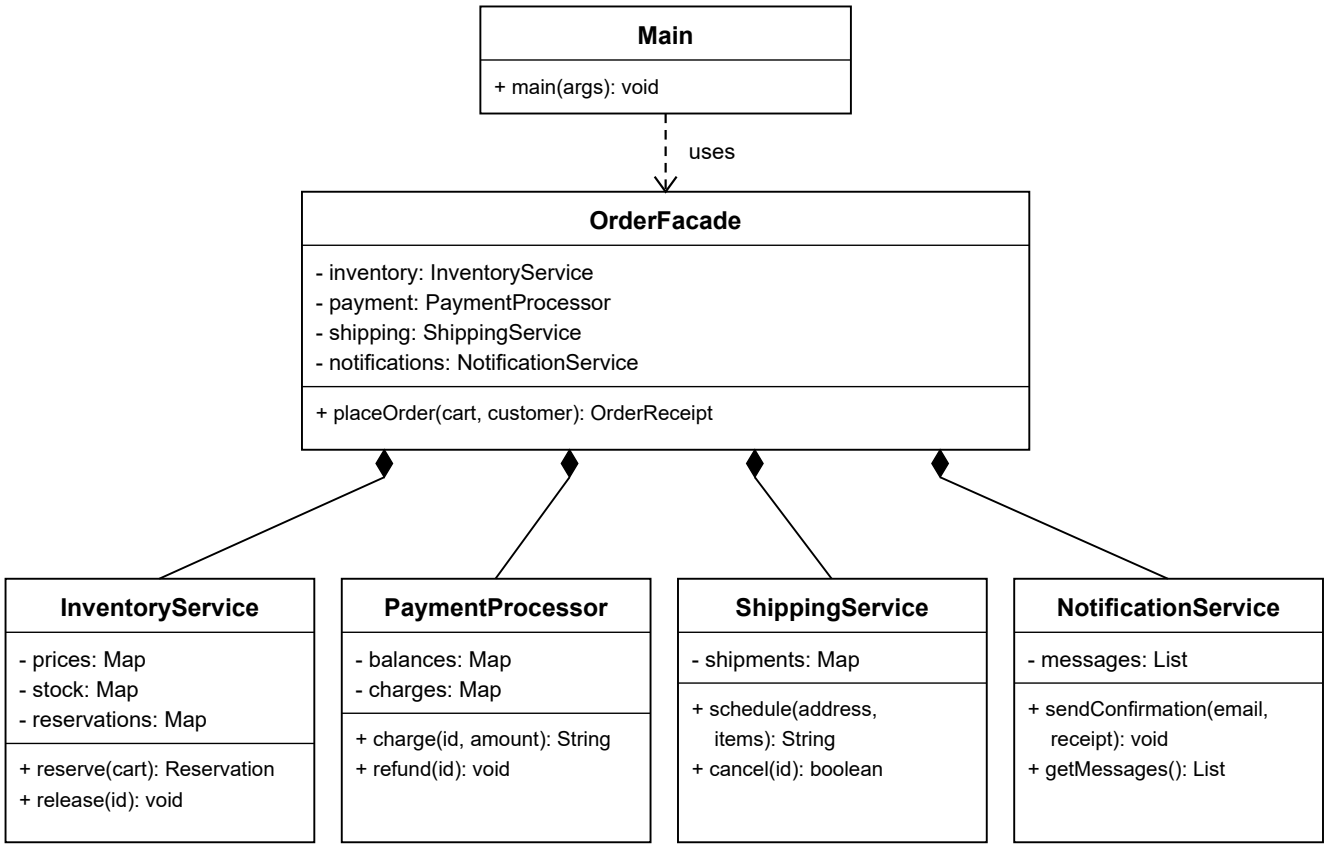


Figure 1 Main depends on the simplified order interface; OrderFacade manages the subsystem.

placeOrder(Map<String, Integer> cart, Customer customer) returns an **OrderReceipt**. Customer holds id, email, and address; OrderReceipt holds orderId, total, and shipmentId. The internal Reservation holds its ID and total. These data records are omitted from the diagram to keep its pattern roles clear.

5 Successful order flow

Reserve stock, charge payment, schedule delivery, then confirm. Inventory checks the whole cart and calculates the total before reducing stock. Payment debits that total. Shipping saves the address and item snapshot and returns its ID. Notification queues a message from the receipt. Only then does the facade return the receipt to the client.

6 Failure and recovery

Inventory validation happens before mutation, so a rejected cart needs no compensation. After a successful reservation, payment, shipping, or notification may fail. The facade then cancels any shipment, refunds any completed charge, and releases the reservation, in reverse order. Missing operation IDs are safe no-ops. The caller receives an IllegalStateException with the original failure preserved as its cause.

7 Clean Code principles

The following six principles are demonstrated by annotated excerpts from the implemented project. References name the source file and its actual line numbers.

7 1 Meaningful names

The public method and parameter names describe the business request. The caller can read placeOrder, cart, and customer without knowing how any service works.

OrderFacade.java, lines 13-15

```
public OrderReceipt placeOrder(Map<String, Integer> cart, Customer customer) {
    if (customer == null || cart == null) {
        throw new IllegalArgumentException("Customer and cart are required.");
```

Annotation. The method names an order operation instead of a vague action such as process or execute. The validation message also states exactly which inputs are missing.

7 2 Single responsibility through delegation

The facade is responsible for coordination. Calculation, balance management, address validation, and message composition remain in their respective services.

OrderFacade.java, lines 23-28

```
paymentId = payment.charge(customer.id(), reservation.total());
shipmentId = shipping.schedule(customer.address(), items);
OrderReceipt receipt = new OrderReceipt(
        "ORD-" + nextOrderId++, reservation.total(), shipmentId);
notifications.sendConfirmation(customer.email(), receipt);
return receipt;
```

Annotation. These statements order service calls and pass results between them. There is no price loop, balance arithmetic, or email formatting inside the facade. For example, a price calculation change belongs in InventoryService.

7 3 A small API and least knowledge

The application client interacts with one facade and ordinary business data. It does not obtain service objects or pass a reservation or payment token.

Main.java, lines 20-23

```
OrderReceipt receipt = shop.placeOrder(cart, alice);
System.out.println("Order: " + receipt.orderId());
System.out.println("Total: " + receipt.total() + " USD");
System.out.println("Shipment: " + receipt.shipmentId());
```

Annotation. One call replaces a sequence of subsystem calls. The result exposes business facts, including a shipment tracking ID, while internal reservation and payment IDs stay out of the public result. This follows the least knowledge principle discussed in Lecture 6 [2].

7 Clean Code principles continued

7 4 Small cohesive classes

PaymentProcessor handles account charges and refunds. Its guard clauses keep payment rules together and reject invalid operations before the balance changes.

PaymentProcessor.java, lines 15-23

```
if (amount == null || amount.signum() <= 0) {
    throw new IllegalArgumentException("Payment amount must be positive.");
}
BigDecimal balance = balances.get(customerId);
if (balance == null) {
    throw new IllegalArgumentException("Unknown payment account: " + customerId);
}
if (balance.compareTo(amount) < 0) {
    throw new IllegalStateException("Payment declined: insufficient balance.");
```

Annotation. The rules concern only payment amounts, account existence, and funds. The class does not create shipments or change product quantities. The same separation holds for the other three services.

7 5 Explicit error handling and rollback

The facade contains the recovery sequence in one place. It reverses completed steps and keeps the original cause available to the caller.

OrderFacade.java, lines 29-33

```
} catch (IllegalArgumentException | IllegalStateException failure) {
    shipping.cancel(shipmentId);
    payment.refund(paymentId);
    inventory.release(reservation.id());
    throw new IllegalStateException("Order failed: " + failure.getMessage(), failure);
```

Annotation. A failed notification can happen after both payment and shipping. Cancellation, refund, and release therefore run in reverse order. Refund and release remove their saved operation record before restoring state, preventing a second call from adding money or stock twice. Successful notifications are the final step.

7 6 Protect mutable state

Subsystem services keep their maps and lists private and copy shared collections. The outbox reader returns an unmodifiable snapshot.

NotificationService.java, lines 19-20

```
public List<String> getMessages() {
    return List.copyOf(messages);
```

Annotation. A caller cannot delete stored notifications through the returned list. Inventory and shipping also use Map.copyOf when saving items, so later changes to a caller-owned cart do not alter an existing reservation or shipment. The facade starts checkout with a separate copy of the supplied cart.

These boundaries simplify changes without making services inaccessible. All four service classes remain public. Tests demonstrate direct service use, while Main consistently uses the facade. No interface hierarchy, factory, singleton, or second facade is added.

8 Verification

The complete source was compiled with JDK 17 and all compiler lint checks enabled. There were no warnings. The standalone test runner reported **All 15 tests passed.** The tests are independent and require no network or external test framework.

Test coverage	Observed result
Total and successful charge	A keyboard and mouse cost 75.00 USD; later purchases reflect the reduced balance.
Declined payment	Reserved keyboards become available to Alice after Bob cannot pay.
Shipping and notification failure	A subsequent 100.00 USD purchase succeeds, proving both the balance and stock were restored.
Invalid cart and unknown account	Unknown products, unavailable stock, invalid quantities, empty carts, missing inputs, and unknown accounts are rejected.
Direct subsystem operations	Release and refund restore state once; shipping cancellation removes the saved record; outbox snapshots cannot be changed.

Run the demonstration and tests from the repository root:

```
javac --release 17 -Xlint:all -d out src/shop/*.java test/shop/*.java
java -cp out shop.Main
java -cp out shop.OrderFacadeTest
```

The demo first rejects Bob's two-keyboard purchase, then completes Alice's purchase of those same two keyboards for 100.00 USD, and finally rejects another keyboard order because no keyboards remain. The captured output is in docs/verification.txt.

9 Conclusion and limitations

The facade hides service creation, the checkout order, handoff of reservation and payment results, and compensation after a failure. It simplifies use of the subsystem while leaving its full service APIs directly accessible to advanced callers. The client only needs a cart, a customer, and one placeOrder call.

This is a single-threaded, in-memory simulation: it does not send real email, contact a bank or carrier, or persist data. Email validation is basic and IDs are local to an instance. Compensation is not a database transaction. Treating a notification failure as a failed order is a deliberate rule for this example; a production shop might retry notification instead. Keeping one checkout operation also avoids turning the facade into a class responsible for the entire application.

10 Repository and references

GitHub repository
https://github.com/aokoshi/SDP-Assignment4

[1] Assignment 4 Facade Pattern Instructions. Provided course assignment document. Option B, technical requirements, Clean Code requirements, and report criteria.
[2] Almas Ospanov. Lecture 6, The Facade Design Pattern. Provided lecture slides, especially pages 10, 11, 16, 19, and 20.